

[2026-1] Machine Learning Assignment # 1

개인 홈 네트워크 침입 및 사생활 유출 위험 탐지를 위한
악성 트래픽 분류 모델 구축

- 과 목: Machine Learning
- 학 과: 인공지능 공학과
- 학 번: 12223560
- 이 름: 윤병우
- 제출일: 2026.05.

목 차

1. 서론	3
1.1 문제 배경	3
1.2 기존 한계	3
2. 문제 정의	3
2.1 해결 목표	3
2.2 활용 가능 사용자 및 기대 효과	4
2.3 연구 질문(RQ)	4
3. 데이터셋 및 데이터 구성	4
3.1 데이터셋 소개	4
3.2 Feature 및 Label 구성	5
3.3 Train / Validation / Test Split	6
3.4 데이터 불균형 여부	6
4. Baseline 모델 (KNN)	6
4.1 Baseline 모델 선정	6
4.2 Sample 데이터셋 사용	6
4.3 Hyperparameter Tuning	7
4.4 Baseline 실험결과	7
5. 메인 모델 (Random Forest)	8
5.1 모델 선정	8
5.2 Hyperparameter Tuning	9
5.3 Sample 데이터셋 실험 결과	11
5.4 전체 데이터셋 기반 메인 모델 평가	12
5.5 Feature Importance	13
6. 종합 비교 및 분석	13
6.1 KNN과 Random Forest 성능 비교 분석	13
6.2 오분류 사례 및 데이터셋 특성 분석	14
7. 결론	15

1. 서론

1.1 문제 배경

최근 IP카메라, 스마트 월패드, IoT 가전 등 가정용 스마트 기기의 보급과 활용이 증가하면서, 이를 대상으로 한 외부 해킹 및 사생활 유출에 대한 우려가 커지고 있다. 다양한 IoT 기기들의 사용이 늘어나기 시작하며 사용자는 편리한 스마트홈 환경을 이용할 수 있게 되었다. 하지만 동시에 외부 공격자가 침입할 수 있는 지점도 함께 늘어나게 되었고 보안 취약점을 이용한 원격 침입 및 개인정보 유출 위험성 또한 증가하고 있는 상황이다.

특히, IP카메라나 월패드처럼 사생활 정보와 직접적으로 연결된 기기는 해킹이 발생했을 때 단순한 인터넷 장애가 아니라 영상 유출, 원격 감시, 개인정보 침해와 같은 문제로 이어질 수 있다. 하지만 일반적인 가정 환경에서는 기업 보안망과 달리 전문적인 관리가 어렵고 네트워크 로그나 비정상 트래픽을 직접 분석할 수 있는 사용자도 거의 없다. 이로 인해 실제 공격이 발생하더라도 이를 즉시 인지하거나 대응하기 힘든 상황이다.

또한 일반 사용자들 중 반복적인 보안 시스템 오작동으로 인해 정상적인 인터넷 사용 환경에 불편함을 느껴 보안 기능 자체를 비활성화하는 경우도 존재한다. 따라서 홈 네트워크 환경에서는 단순히 탐지 성능만 높은 시스템이 아니라, 사용자 편의성을 최대한 유지하면서도 실제 악성 트래픽을 효과적으로 탐지할 수 있는 보안 시스템이 필요하다.

1.2 기존 한계

기존 공유기 및 보안 시스템에 탑재된 규칙 기반 탐지 방식은 사전에 정의된 패턴만을 기준으로 동작하기 때문에 새로운 형태의 공격이나 변칙적인 악성 트래픽에 유연하게 대응하기 어렵다. 또한 온라인 게임, 영상 스트리밍 등 정상적인 상황에서도 이를 비정상 트래픽으로 잘못 탐지하는 오탐(False Positive, FP) 문제가 자주 발생한다.

반대로 오탐을 줄이기 위해 탐지 기준을 과도하게 완화할 경우 실제 악성 트래픽을 놓치는 미탐(False Negative, FN) 문제가 발생할 수 있으며, 이는 사생활 유출이나 원격 기기 탈취와 같은 심각한 피해로 이어질 가능성이 존재한다. 결국 기존 방식은 사용자 편의성을 높이면 보안성이 낮아지고, 보안성을 높이면 정상 사용 환경이 방해받을 수 있다는 한계를 가진다.

2. 문제 정의

2.1 해결 목표

본 과제에서는 홈 네트워크 트래픽 데이터를 기반으로 정상 트래픽과 악성 트래픽을 구분하는 머신러닝 기반 침입 탐지 모델을 설계하고자 한다. 단순 규칙 기반 차단 방식이 아니라, 네트워크 트래픽의 특징을 학습하여 정상 트래픽과 악성 트래픽을 구분하는 것이 목표이며 이를 위해 패킷 길이, 프로토콜 유형 등 네트워크 트래픽에서 추출할 수 있는 특징(feature)을 활용한다. 또한 단순 Accuracy 비교만 수행하는 것이 아니라 실제 홈 네트워크 환경에서 중요한 오탐(FP) 및 미탐(FN) 문제를 함께 고려하여 모델의 성능을 분석하고자 한다.

2.2 활용 가능 사용자 및 기대 효과

이 시스템을 통해 이득을 보는 사용자 또는 상황은 다음과 같다.

- ① 보안 설정 및 네트워크 관리에 어려움을 느끼는 일반 홈 네트워크 사용자
: 별도의 전문 지식 없이도 악성 트래픽 및 외부 해킹으로부터 보다 안전한 네트워크 환경을 제공받을 수 있다. 또한 온라인 게임, 실시간 화상통화와 같은 정상적인 인터넷 사용 환경에서 발생하던 기존 시스템의 반복적인 오작동 문제를 줄일 수 있다.
- ② IP카메라 및 스마트홈 기기를 사용하는 일반 가정
: 반려동물 모니터링과 같은 실시간 감시 환경에서 복잡한 보안 설정 없이도 자동화된 보안 보호 효과를 기대할 수 있다. 특히, 사생활 유출과 같은 위험 요소를 줄이면서도 정상적인 실시간 스트리밍 환경은 유지할 수 있다는 이점을 갖는다.

2.3 연구 질문(RQ)

본 과제의 연구 질문(RQ)은 다음과 같다.

“홈 네트워크 트래픽 분류 문제에서 머신러닝 기반 분류 모델이 규칙 기반 탐지 방식의 한계를 개선하면서도 실제 악성 트래픽 탐지 성능을 효과적으로 유지할 수 있는가?”

이를 확인하기 위해 모델의 성능을 단순 Accuracy뿐만 아니라 실제 사용자 환경에서 중요한 FPR, Recall, Precision 등을 함께 분석함으로써 사용자 편의성과 보안성을 동시에 만족할 수 있는지 확인하고자 한다.

3. 데이터셋 및 데이터 구성

3.1 데이터셋 소개

본 과제에서는 네트워크 트래픽 데이터를 기반으로 정상 트래픽과 악성 트래픽을 분류하는 모델을 학습하기 위해 ‘CICIDS2017’ 데이터셋을 머신러닝 실험에 사용하기 쉽게 정리한 Kaggle의 ‘CICIDS2017: Cleaned & Preprocessed’ 버전을 사용했다. 해당 데이터셋은 Canadian Institute for Cybersecurity에서 제공한 침입 탐지 평가용 데이터셋으로 Normal Traffic과 DoS, DDoS, Port Scanning 등 다양한 공격 트래픽을 포함하고 있다. 또한 네트워크 패킷 원본을 직접 사용하는 것이 아니라 flow 단위의 feature와 label이 CSV 형태로 제공되어 KNN, Random Forest와 같은 머신러닝 분류 모델을 적용하기에 적합하다.

해당 데이터셋은 정상 트래픽과 여러 공격 유형의 트래픽을 함께 포함하고 있으므로, 본 과제에서 설정한 정상/악성 트래픽 분류 문제를 실험하기에 적절하다고 판단했다. 단순히 전체 정확도만 확인하는 것이 아니라, 정상 트래픽을 공격으로 잘못 판단하는 오탐과 실제 공격 트래픽을 놓치는 미탐을 함께 분석할 수 있다는 점에서 본 과제의 연구 목적과도 연결된다.

3.2 Feature 및 Label 구성

입력 feature는 트래픽의 지속 시간, 패킷 수, 전송량, 패킷 길이, 트래픽 발생 간격 등을 나타내는 값들로 구성되어 있으며 주요 feature는 다음과 같다.

Feature	의미
Destination Port	목적지 포트 번호
Flow Duration	하나의 flow가 지속된 시간
Total Fwd Packets	Forward 방향으로 전송된 전체 패킷 수
Total Length of Fwd Packets	Forward 방향 패킷들의 총 길이
Flow Bytes/s	초당 전송 바이트 수
Flow Packets/s	초당 전송 패킷 수
Packet Length Mean	패킷 길이의 평균값
Packet Length Std	패킷 길이의 표준편차
Idle Mean	Flow 내 평균 유휴 시간

표 1 Feature

Label은 데이터셋의 ‘Attack Type’ 컬럼을 기준으로 구성하였다. 원래 데이터에는 Normal Traffic, Dos, DDoS, Port Scanning, Brute Force, Web Attacks, Bots와 같은 여러 트래픽 유형이 존재한다. 하지만 본 과제의 목표는 공격 유형을 세부적으로 분류하는 것이 아니라 정상 트래픽과 악성 트래픽을 구분하는 것이기 때문에 기존의 다중 클래스 label을 다음과 같이 이진 label로 변환하였다.

기존 Attack Type	변환 Label	의미
Normal Traffic	0	정상 트래픽
DoS	1	악성 트래픽
DDoS	1	악성 트래픽
Port Scanning	1	악성 트래픽
Brute Force	1	악성 트래픽
Web Attacks	1	악성 트래픽
Bots	1	악성 트래픽

표 2 Label 변환

이와 같이 Label을 구성함으로써 문제를 정상 트래픽과 악성 트래픽을 구분하는 binary classification 문제로 정의하였다. 모델 학습에서는 변환된 이진 label을 정답값으로 사용하고 feature들을 입력값으로 사용했다.

3.3 Train / Validation / Test Split

모델의 학습 및 평가를 위한 데이터 분할 비율은 train : validation : test = 70 : 15 : 15로 설정하였다. 해당 데이터셋은 정상 트래픽과 악성 트래픽의 개수 차이가 존재하기 때문에 단순 랜덤 분할만 수행할 경우 각 set의 label 비율이 달라질 수 있다. 이러한 문제를 줄이기 위해 stratified split을 적용하여 train, validation, test set이 원본 데이터와 유사한 정상/악성 트래픽 비율을 유지하도록 했다. 이를 통해 특정 set에 정상 또는 악성 트래픽이 과도하게 포함되는 상황을 방지하고자 했다.

3.4 데이터 불균형 여부

Attack Type 칼럼을 기준으로 이진 label을 구성한 결과, 정상 트래픽은 2,095,057개, 악성 트래픽은 425,694개로 나타났다. 즉, 본 데이터셋은 정상 트래픽의 비율이 악성 트래픽보다 높은 불균형 데이터셋이다.

Class	Label	Count
Normal Traffic	0	2,095,057
Attack Traffic	1	425,694

표 3 Data 분포

본 과제에서는 이러한 데이터 불균형 문제를 조정하지 않고 원래의 분포를 유지하였다. 이는 홈 네트워크 환경에서도 대부분의 트래픽은 정상적인 사용 과정에서 발생하며 악성 트래픽이 상대적으로 적게 발생한다는 문제 설정과 연결된다.

4. Baseline 모델 (KNN)

4.1 Baseline 모델 선정

비교 기준이 되는 baseline 모델로 K-Nearest-Neighbors(KNN)을 사용하였다. KNN은 입력 데이터와 가장 가까운 이웃 데이터들의 label을 기반으로 클래스를 분류하는 거리 기반 머신러닝 모델이다. 모델 구조가 비교적 단순하고 직관적이기 때문에 baseline 모델로 적절하다고 판단하였다.

4.2 Sample 데이터셋 사용

본 과제에서 사용하는 CICIDS2017 데이터셋은 전체 데이터 수가 매우 크다. 큰 데이터셋에서 KNN의 연산 비용이 크다는 점을 고려하여 baseline 단계에서는 stratified sampling을 적용한 sample 데이터셋을 사용하였다. 이 과정에서 원본 데이터의 정상/공격 비율이 최대한 유지되도록 구성하여 데이터 분포 특성을 보존하고자 했으며 별도의 feature scaling은 적용하지 않았다. 사용한 sample 데이터셋의 크기는 train sample 100,000개, validation sample 30,000개, test sample 30,000개이다.

4.3 Hyperparameter Tuning

KNN에서 가장 중요한 hyperparameter는 이웃 데이터의 개수를 의미하는 k 값이다. 본 실험에서는 k 값을 3, 5, 7, 9, 11로 설정하여 validation sample에서 성능을 비교하였다. 이때, F1-score를 기준으로 최고의 k 값을 선택하였으며 이는 FP(False Positive)와 FN(False Negative)을 함께 고려하기 위함이다. 이후 validation 단계에서 선택된 최적의 k 값을 기반으로 test sample 데이터에서 baseline 모델의 최종 성능을 평가하였다.

4.4 Baseline 실험결과

먼저 validation sample에서 k값에 따른 성능을 비교하였다. 실험 결과 k=3에서 가장 높은 F1-score를 기록하였으며, k 값이 증가할수록 Recall과 F1-score가 소폭 감소하는 경향을 보였다. 따라서 성능을 가장 안정적으로 유지한 k=3을 최종 hyperparameter로 선택하였다.

	Model	Accuracy	Precision	Recall	F1-score	FPR	FP	FN
0	KNN (k=3)	0.9855	0.9511	0.9639	0.9575	0.0101	251	183
1	KNN (k=5)	0.9839	0.9440	0.9619	0.9529	0.0116	289	193
2	KNN (k=7)	0.9823	0.9383	0.9582	0.9481	0.0128	319	212
3	KNN (k=9)	0.9811	0.9359	0.9534	0.9446	0.0133	331	236
4	KNN (k=11)	0.9800	0.9331	0.9497	0.9413	0.0138	345	255

그림 1 KNN Validation 성능 비교

또한 k 값 변화에 따른 성능 차이를 시각적으로 확인하기 위해 validation 성능 그래프를 함께 분석하였다.

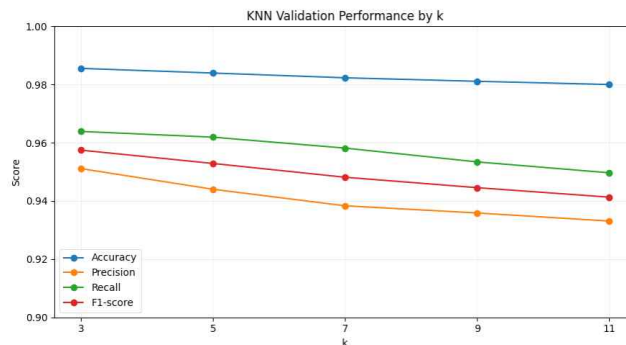


그림 2 KNN Validation 성능 그래프

Validation 단계에서 선택된 k=3 모델을 기반으로 test sample에서 최종 성능을 평가하였다. 실험 결과 baseline 모델은 Accuracy 0.9858, F1-score 0.9584를 기록하였으며 FP와 FN이 일부 발생했다. 이는 대부분의 정상 트래픽을 올바르게 분류하였으나, 일부 정상 트래픽이 공격으로 분류되는 오탐(FP)과 공격 트래픽이 정상으로 분류되는 미탐(FN)이 함께 발생하였음을 보여준다.

	Model	Accuracy	Precision	Recall	F1-score	FPR	FP	FN
0	KNN (k=3)	0.9858	0.9512	0.9657	0.9584	0.0101	251	174

그림 3 KNN Baseline 최종 성능

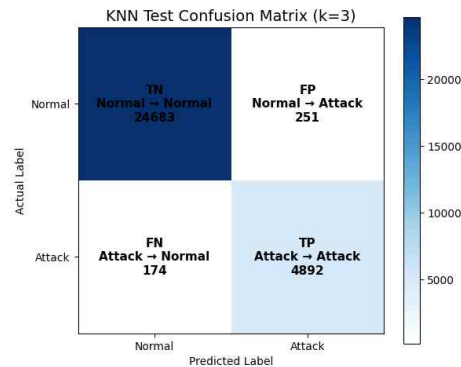


그림 4 KNN Confusion Matrix

이러한 결과를 통해 KNN 모델은 기본적인 공격 탐지는 가능하였으나, 데이터 크기가 증가할수록 연산 비용이 커지고 일부 오탐(FP)이 발생하는 한계가 존재함을 확인할 수 있었다. 따라서 본 과제에서는 보다 안정적인 분류 성능과 높은 확장성을 제공할 수 있는 Random Forest 모델을 메인모델로서 적용하고자 한다.

5. 메인 모델 (Random Forest)

5.1 모델 선정

본 과제에서는 메인 모델로 Random Forest(RF)를 사용하였다. Random Forest는 여러 개의 Decision Tree를 생성한 뒤 각 트리의 예측 결과를 종합하여 최종 클래스를 결정하는 ensemble 기반 머신러닝 모델이다.

KNN baseline 실험 결과, 비교적 높은 성능을 보였지만 데이터 수가 증가할수록 추론 과정에서 많은 계산량이 필요하며 일부 FN(False Negative)과 FP(False Positive)가 발생하였다. 또한 KNN은 데이터 간 거리 계산에 의존하기 때문에 데이터 규모가 커질수록 효율성이 저하될 수 있다.

반면 Random Forest는 다수의 Decision Tree를 기반으로 동작하기 때문에 보다 안정적인 분류 성능을 제공할 수 있으며 복잡한 비선형 패턴 학습에도 유리하다. 또한 Tree 기반 모델 특성 상 feature importance 분석이 가능하다는 장점이 있다. 이러한 이유로 본 과제에서는 Random Forest를 최종 메인 모델로 선정했다.

5.2 Hyperparameter Tuning

Random Forest의 성능을 최적화하기 위해 주요 hyperparameter tuning을 수행하였다. 실험은 sample 데이터셋을 기반으로 진행하였으며, validation set의 F1-score를 중심으로 Recall, FP, FN 등을 함께 고려하여 최종 parameter를 선택하였다.

메인 모델의 주요 hyperparameter로는 생성할 트리의 수, Decision Tree의 최대 깊이, 노드를 분할하기 위한 최소 sample 수, leaf node를 구성하기 위한 최소 sample 수 마지막으로 클래스별 가중치를 조정하는 parameter를 hyperparameter로 이용하였다.

(1) n_estimators tuning

n_estimators는 트리 개수가 성능에 미치는 영향을 의미한다. 다른 parameter 값들을 고정하고 트리 개수를 변경하며 성능을 측정했다. 실험 결과 50 이상 구간에서는 전체 성능 차이가 크지 않았으며 트리 개수를 증가시켜도 F1-score 향상이 제한적인 것을 확인할 수 있다.

	n_estimators	Accuracy	Precision	Recall	F1-score	FPR	FP	FN	Time(sec)
0	50	0.9981	0.9966	0.9923	0.9945	0.0007	17	39	1.86
1	150	0.9981	0.9966	0.9921	0.9944	0.0007	17	40	5.74
2	300	0.9981	0.9968	0.9919	0.9944	0.0006	16	41	10.10
3	200	0.9981	0.9968	0.9919	0.9944	0.0006	16	41	6.62
4	500	0.9981	0.9966	0.9919	0.9943	0.0007	17	41	16.93
5	100	0.9980	0.9966	0.9915	0.9941	0.0007	17	43	3.29

그림 5 n_estimators 1차 성능 측정

1차 성능 측정 결과를 바탕으로 50 부근에서 2차 성능 측정을 수행했다. 그 결과 n_estimators=70에서 가장 높은 F1-score를 기록하였다. 또한 이후 70 부근에서 3차 성능 측정을 수행하였지만 성능 향상이 거의 나타나지 않았기 때문에 최종 값으로 70을 선택하였다.

	n_estimators	Accuracy	Precision	Recall	F1-score	FPR	FP	FN	Time(sec)
0	70	0.9982	0.9968	0.9923	0.9946	0.0006	16	39	1.98
1	50	0.9981	0.9966	0.9923	0.9945	0.0007	17	39	1.55
2	60	0.9981	0.9964	0.9921	0.9943	0.0007	18	40	1.95
3	40	0.9980	0.9964	0.9917	0.9941	0.0007	18	42	1.25
4	30	0.9980	0.9964	0.9915	0.9940	0.0007	18	43	1.08

그림 6 n_estimators 3차 성능 측정

(2) max_depth tuning

max_depth는 트리의 최대 깊이를 의미한다. 해당 parameter에 대해 실험한 결과 트리 최대 깊이에 한계를 두지 않은 None에서 가장 높은 성능이 나타났으며 depth가 증가할수록 성능이 향상되는 경향을 확인할 수 있었다. 그러나 최고 성능이 제한 없음에서 발생했기 때문에 적절한 depth 제한 범위를 추가적으로 확인하기 위해 30 부근을 중심으로 2차 성능 측정을 수행하였다. 이를 통해 depth를 일정 수준까지 증가시키면 성능이 향상되지만 이후 구간에서는 성능 차이가 거의 발생하지 않는다는 점을 확인할 수 있었다.

	max_depth	Accuracy	Precision	Recall	F1-score	FPR	FP	FN	Time(sec)
0	35	0.9982	0.9968	0.9923	0.9946	0.0006	16	39	2.08
1	None	0.9982	0.9968	0.9923	0.9946	0.0006	16	39	1.98
2	40	0.9982	0.9968	0.9923	0.9946	0.0006	16	39	2.05
3	30	0.9981	0.9966	0.9921	0.9944	0.0007	17	40	2.22
4	25	0.9980	0.9966	0.9917	0.9942	0.0007	17	42	2.19

그림 7 max_depth 2차 성능 측정

실험 결과 max_depth=None, 35, 40에서 유사한 성능이 나타났다. 그러나 무제한 깊이보다 트리 깊이를 제한한 max_depth=35가 과도한 tree 성장을 방지하면서도 동일한 수준의 성능을 유지할 수 있다고 판단하여 최종값으로 선택하였다.

(3) min_samples_split tuning

min_samples_split은 노드를 분할하기 위한 최소 sample 수를 의미한다. 다양한 split 기준에 대한 1차 성능 측정을 수행한 결과 min_samples_split=5 부근에서 가장 높은 성능이 나타나는 것을 확인할 수 있었다. 그러나 각 설정 성능 차이가 크지 않았기 때문에, 최적값을 보다 정확하게 확인하기 위해 5 주변 구간에 대한 추가적인 2차 성능 측정을 수행하였다.

	min_samples_split	Accuracy	Precision	Recall	F1-score	FPR	FP	FN	Time(sec)
0	5	0.9983	0.9966	0.9935	0.9951	0.0007	17	33	2.01
1	6	0.9983	0.9966	0.9931	0.9949	0.0007	17	35	2.13
2	4	0.9983	0.9968	0.9929	0.9949	0.0006	16	36	2.15
3	3	0.9982	0.9966	0.9929	0.9948	0.0007	17	36	2.14
4	7	0.9981	0.9964	0.9925	0.9945	0.0007	18	38	1.99

그림 8 min_samples_split 2차 성능 측정

실험 결과 min_samples_split=5에서 가장 높은 Recall 및 F1-score를 기록하였으며 FN 또한 가장 적게 발생하는 것을 확인하여 min_samples_split=5를 최종값으로 선택하였다.

(4) min_samples_leaf tuning

min_samples_leaf는 leaf node를 구성하기 위한 최소 sample 수를 의미한다. 해당 parameter에 대해 실험한 결과 min_samples_leaf 값이 증가할수록 Recall 및 F1-score가 감소하는 경향을 보였다.

	min_samples_leaf	Accuracy	Precision	Recall	F1-score	FPR	FP	FN	Time(sec)
0	1	0.9983	0.9966	0.9935	0.9951	0.0007	17	33	2.01
1	2	0.9981	0.9964	0.9923	0.9944	0.0007	18	39	2.01
2	4	0.9978	0.9958	0.9913	0.9936	0.0008	21	44	1.88
3	8	0.9974	0.9954	0.9893	0.9924	0.0009	23	54	2.01
4	16	0.9968	0.9954	0.9856	0.9905	0.0009	23	73	1.88

그림 9 min_samples_leaf 1차 성능 측정

이는 leaf node 크기가 커질수록 세밀한 패턴 학습이 어려워졌기 때문으로 해석할 수 있다. 따라서 가장 높은 성능을 기록한 min_samples_leaf=1을 최종값으로 선택하였다.

(5) class_weight tuning

마지막으로 데이터셋의 불균형 문제를 고려하여 class_weight parameter를 비교하였다.

	class_weight	Accuracy	Precision	Recall	F1-score	FPR	FP	FN	Time(sec)
0	balanced	0.9985	0.9966	0.9945	0.9956	0.0007	17	28	1.69
1	None	0.9983	0.9966	0.9935	0.9951	0.0007	17	33	2.19

그림 10 class_weight 1차 성능 측정

실험 결과 class_weight="balanced" 적용 시 Recall 및 F1-score가 향상되었으며, FN 또한 감소하는 것을 확인하였다. 따라서 클래스의 불균형 문제를 고려해 메인 모델에 balanced를 적용하였다.

최종적으로 선택된 hyperparameter는 다음과 같다.

	Hyperparameter	Selected Value
0	n_estimators	70
1	max_depth	35
2	min_samples_split	5
3	min_samples_leaf	1
4	class_weight	balanced

그림 11 최종 hyperparameter

5.3 Sample 데이터셋 실험 결과

최종적으로 선택된 Hyperparameter를 적용한 Random Forest 모델을 sample dataset에서 학습한 뒤 sample test set에서 평가하였다. 실험 결과 Random Forest는 매우 높은 F1-score와 Accuracy를 기록하였으며 Recall 또한 높은 수준으로 나타났다. 그리고 FP와 FN 모두 낮은 수준으로 유지되는 것을 확인할 수 있었다.

	Model	Accuracy	Precision	Recall	F1-score	FPR	FP	FN
0	Random Forest	0.9986	0.9953	0.9964	0.9959	0.0010	24	18

그림 12 Sample 데이터셋 실험 결과

Confusion Matrix 결과에서도 대부분의 공격 트래픽과 정상 트래픽을 정확하게 분류하고 있다는 것을 확인할 수 있다. 특히, 공격 트래픽을 정상 트래픽으로 잘못 분류하는 FN이 18건 수준으로 나타났다. 정상 트래픽을 공격으로 잘못 탐지하는 비율인 FP 역시 24건으로 나타났다. 이는 전체 test sample 규모를 고려했을 때 비교적 낮은 수준의 탐지 실패율이라고 볼 수 있고 Random Forest 모델이 정상 트래픽과 공격 트래픽의 패턴을 비교적 안정적으로 학습하였음을 의미한다.

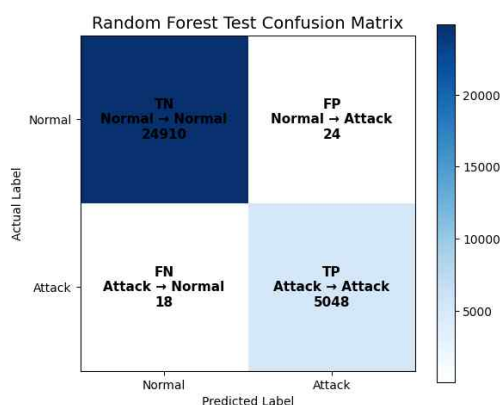


그림 13 Confusion Matrix

실험 결과를 종합하면, tuning된 Random Forest 모델은 sample dataset 기준에서 매우 높은 공격 탐지 성능을 보였으며, 특히 낮은 FN을 유지하면서도 높은 Recall과 F1-score를 기록하였다. 이를 통해 Random Forest가 홈 네트워크 환경의 악성 트래픽 탐지 문제에 효과적으로 적용될 가능성을 확인할 수 있었다.

5.4 전체 데이터셋 기반 메인 모델 평가

Sample dataset에서 수행한 hyperparameter tuning 결과를 바탕으로 최종 Random Forest 모델을 구성하였다. 이후 해당 설정을 그대로 사용하여 전체 dataset으로 모델을 다시 학습하고 최종 성능을 평가하였다. 이 과정은 sample 기반 실험에서 확인한 성능이 전체 데이터 환경에서도 유지되는지 확인하기 위한 최종 검증 단계이다.

전체 데이터셋 기반 성능 측정 결과 Random Forest 모델은 Accuracy 0.9988, F1-score 0.9964를 기록하였다. 또한 공격 트래픽 탐지와 직접적으로 관련된 Recall은 0.9966으로 나타났다으며, FPR은 0.0008로 매우 낮게 유지되었다.

	Model	Accuracy	Precision	Recall	F1-score	FPR	FP	FN
0	Random Forest Full	0.9988	0.9961	0.9966	0.9964	0.0008	249	216

그림 14 전체 데이터셋 성능 평가표

Confusion Matrix를 보면 정상 트래픽 314,010개를 올바르게 분류하였고 정상 트래픽을 공격으로 잘못 분류한 FP는 249건으로 나타났다. 공격 트래픽의 경우 63,638개를 올바르게 탐지하였으며 공격 트래픽을 정상으로 잘못 분류한 FN은 216건이었다.

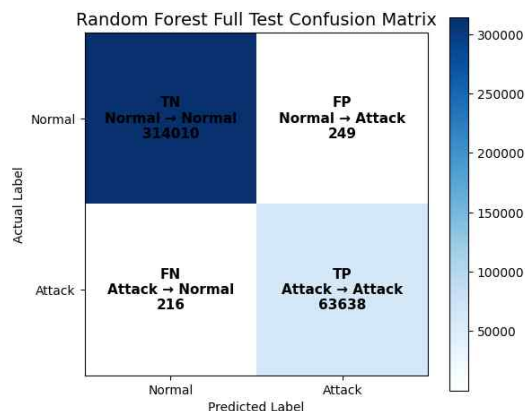


그림 15 Confusion Matrix

이 결과는 sample 데이터셋에서 tuning한 Random Forest 모델이 전체 데이터셋에서도 성능을 유지했음을 보여준다. 특히 데이터셋 규모가 커졌음에도 Recall과 F1-score가 높은 수준으로 유지되었고 FP와 FN도 낮은 비율로 나타났다. 따라서 최종 Random Forest 모델은 전체 데이터 환경에서도 안정적인 악성 트래픽 분류 성능을 보였다고 판단할 수 있다.

5.5 Feature Importance

전체 데이터셋으로 학습한 최종 Random Forest 모델을 대상으로 feature importance를 분석하였다. Feature importance는 모델이 정상 트래픽과 공격 트래픽을 구분하는 과정에서 어떤 feature를 중요하게 활용했는지 확인하기 위한 것이다. 이를 통해 단순 성능 비교뿐만 아니라 모델이 실제로 어떤 네트워크 트래픽 특성을 기반으로 공격 여부를 판단했는지 분석하고자 하였다. 분석 결과 패킷 길이 관련 feature들이 높은 중요도를 가지는 것으로 나타났다. 특히 Packet Length Variance, Bwd Packet Length Std, Average Packet Size, Packet Length Std 등이 상위 importance를 기록하였다. 이는 공격 트래픽이 정상 트래픽과 비교했을 때 패킷 크기와 패킷 분포 측면에서 차이를 보였음을 의미한다. 또한 Destination Port가 높은 중요도를 보인 점을 통해 일부 공격 유형은 특정 서비스 포트를 대상으로 반복적인 접근을 시도하는 경우가 많으며 이러한 패턴이 공격 트래픽과 정상 트래픽을 구분하는 과정에 영향을 준 것으로 해석할 수 있다.

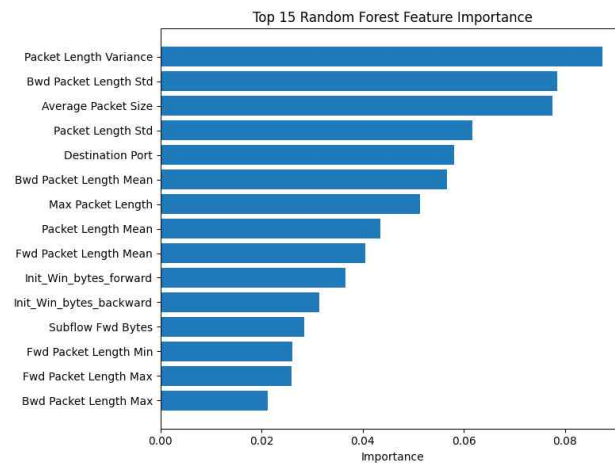


그림 16 Feature Importance 분석 그래프

종합적으로, 최종 Random Forest 모델은 하나의 feature만을 기반으로 예측한 것이 아니라 패킷 길이, 패킷 분포, 포트 정보, TCP 연결 특성 등 다양한 네트워크 트래픽 정보를 복합적으로 활용하여 공격 트래픽을 탐지한 것으로 볼 수 있다. 특히 packet length 관련 feature들의 중요도가 높게 나타난 것은 공격 트래픽이 정상 트래픽과 비교했을 때 비정상적인 흐름 패턴과 패킷 분포 특성을 가진다는 점을 모델이 효과적으로 학습했음을 보여주는 결과라고 할 수 있다.

6. 종합 비교 및 분석

6.1 KNN과 Random Forest 성능 비교 분석

본 과제에서는 KNN을 baseline 모델로 사용하고 Random Forest를 메인 모델로 사용하여 악성 트래픽 분류 성능을 비교하였다. 두 모델은 동일한 sample dataset 환경에서 학습 및 평가하였으며 여러 가지 평가 기준으로 성능을 평가하였다.

	Model	Accuracy	Precision	Recall	F1-score	FPR	FP	FN
0	KNN (k=3)	0.9858	0.9512	0.9657	0.9584	0.0101	251	174
1	Random Forest	0.9986	0.9953	0.9964	0.9959	0.0010	24	18

그림 17 KNN과 Random Forest 비교 표

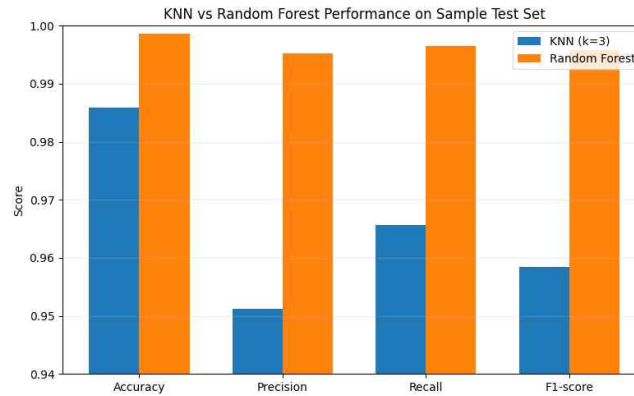


그림 18 KNN과 Random Forest 비교 그래프

실험 결과 Random Forest는 모든 평가 지표에서 KNN보다 높은 성능을 기록하였다. 특히 공격 트래픽을 정상 트래픽으로 잘못 판단하는 FN이 크게 감소한 점이 가장 의미 있는 결과라고 볼 수 있다. 홈 네트워크 환경에서는 실제 공격을 정상으로 판단하는 경우 사생활 유출 등 큰 문제로 이어질 수 있기 때문에 FN 감소는 중요한 의미를 가진다. 또한 KNN은 예측 과정에서 모든 데이터와의 거리 계산이 필요하기 때문에 데이터 규모가 커질수록 연산량이 증가한다. 반면 Random Forest는 학습된 tree 구조를 기반으로 예측을 수행하기 때문에 상대적으로 안정적인 추론이 가능하다. 따라서 Random Forest는 성능뿐만 아니라 실제 네트워크 환경 적용 측면에서도 KNN보다 더 적합한 모델이라고 판단할 수 있다.

6.2 오분류 사례 및 데이터셋 특성 분석

Random Forest 모델은 매우 높은 Accuracy와 F1-score를 기록하였다. 전체 데이터셋 기반 성능 측정 단계에서도 FP와 FN이 모두 낮은 수준으로 나타났으며 대부분의 정상 트래픽과 공격 트래픽을 정확하게 분류하는 것을 확인할 수 있었다. 일부 오분류 사례는 정상 트래픽과 공격 트래픽이 유사하게 나타나는 경우 발생했을 가능성이 있다. 예를 들어 정상 트래픽 중에서도 순간적으로 packet flow가 집중되거나 비정상적인 packet length distribution이 나타나는 경우 공격 트래픽과 유사하게 판단될 수 있다. 반대로 일부 공격 트래픽은 정상 트래픽과 유사한 flow pattern을 가지는 경우 정상으로 분류되었을 가능성이 있다. 다만 성능이 너무 높기 때문에 단순히 오분류 사례 자체보다 왜 이렇게 높은 성능이 나타났는지에 대한 분석이 중요하다고 볼 수 있다. 본 과제에서 사용한 'CICIDS2017 cleaned dataset'은 이미 전처리가 완료된 형태의 데이터셋이며 일부 feature가 label과 강한 상관관계를 가지도록 구성되었을 가능성이 존재한다. 또한 train과 test가 동일한 환경에서 수집된 traffic pattern을 공유하기 때문에 모델이 유사한 패턴을 효과적으로 학습했을 가능성도 함께 고려할 필요가 있다. 즉, 실험 결과는 Random Forest가 네트워크 트래픽 기반 악성 행위 탐지 문제에서 높은 성능을 보일 수 있음을 보여주지만 동시에 이러한 높은 성능이 데이터셋의 전처리 특성 및 train/test 환경 유사성의 영향을 일부 받았을 가능성도 함께 고려할 필요가 있음을 보여준다.

7. 결론

본 과제에서는 홈 네트워크 환경에서 발생할 수 있는 악성 트래픽 및 사생활 유출 위험을 탐지하기 위해 머신러닝 기반 트래픽 분류 모델을 구축하였다. Baseline 모델로 KNN을 사용하였으며, 메인 모델로 Random Forest를 적용하여 성능을 비교하였다.

실험 결과 Random Forest는 KNN보다 모든 평가 지표에서 더 높은 성능을 기록하였다. 특히 공격 트래픽을 정상 트래픽으로 잘못 판단하는 FN을 크게 감소시켰으며 이는 실제 공격 탐지 안정성을 높였다는 점에서 중요한 의미를 가진다. 또한 feature importance 분석 결과를 통해 Random Forest가 packet length, port 정보, TCP 연결 특성 등 다양한 네트워크 트래픽 패턴을 효과적으로 학습했음을 확인할 수 있었다.

다만 본 실험은 CICIDS2017 cleaned dataset 기반으로 수행되었기 때문에 데이터셋의 전체 리 특성이나 train/test 환경 유사성의 영향을 일부 받았을 가능성이 존재한다. 따라서 현재 결과를 실제 홈 네트워크 환경에 그대로 일반화하기에는 한계가 있으며 다양한 환경 기반 추가 검증이 필요하다. 향후에는 최신 공격 유형이 포함된 데이터셋을 추가적으로 활용하고 다양한 ensemble 기반 모델과의 비교 실험을 통해 보다 안정적인 악성 트래픽 탐지 모델로 확장하고자 한다.

8. 데이터 셋 출처 및 인공지능 사용 부분

8.1 데이터셋 출처

Kaggle, "CICIDS2017: Cleaned & Preprocessed Dataset." Available:
<https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset>

8.2 인공지능 사용 부분

해당 프로젝트에서는 hyperparameter 설정 방법과 같은 방법론적인 부분을 제외한 실험의 전반적인 코드 작성을 위해 생성형 AI(ChatGPT)를 사용하였다. 그리고 생성형 AI를 이용해 최종 보고서에 대해 전체적인 구조나 내용 측면에서 추가할 부분 혹은 수정하면 좋을 부분이 없는지 확인하고 실제로 수정했을 때 더 낫다고 판단되는 부분을 수정했다. 또한 GIT에 올릴 README 파일 작성에 활용하였다.